

Technische Hochschule Würzburg-Schweinfurt
Fakultät Informatik und Wirtschaftsinformatik

Projektdokumentation

Workflowbasierte Rückbestätigung von Lieferterminen

vorgelegt an der Technischen Hochschule Würzburg-Schweinfurt
in der Fakultät Informatik und Wirtschaftsinformatik
im Rahmen der Projektarbeit

Name 1
Name 2
Name 3

Betreuerin: Frau Prof. Dr. Saueressig

Technischer Betreuer: Herr Prof. Dr. Braun

Abgabedatum: 16.09.2026

Inhaltsverzeichnis

1	Einleitung	4
1.1	Motivation und Problemstellung	4
1.2	Zielsetzung	4
1.3	Methodische Vorgehensweise	4
2	IST-Analyse	4
2.1	Bestehender Avisierungsprozess	4
2.2	Bestehende Kommunikationskanäle	4
2.3	Schwachstellen und Optimierungspotenziale	4
2.4	Funktionale Anforderungen	5
2.5	Nichtfunktionale Anforderungen	5
3	SOLL-Konzept	5
3.1	Zielbild des Avisierungsprozesses	5
3.2	SOLL-Prozess	5
3.2.1	Prozessablauf	5
3.2.2	BPMN-Modell	6
4	Systemarchitektur	7
4.1	Gesamtarchitektur	7
4.2	Systemkomponenten	7
4.3	Schnittstellen und Interaktionen	8
5	Backend des Avisierungsservice – Konzeption und Implementierung	10
5.1	Architektur- und Domänenentwurf	10
5.1.1	Architekturprinzipien, Verantwortungsgrenzen und Abhängigkeits- richtung	10
5.1.2	Domänen- und Zustandsmodell	10
5.1.3	Persistenzmodell und Schemaentwicklung	10
5.2	Technische Umsetzung des Avisierungs- und Rückbestätigungsprozesses . .	10
5.2.1	Übernahme von Avisierungsaufträgen, Validierung und Prozessstart	10
5.2.2	Dauerhafte Workflow-Steuerung mit Camunda	10
5.2.3	Kundenreaktion und Fristablauf	10
5.2.4	Erneute Avisierung und Ablösung veralteter Anfragen	10
5.2.5	Statusrückmeldung an DISPO	10
5.2.6	Fehlerbehandlung, Wiederholungen, Nebenläufigkeit und Idempoten- zenzgrenzen	10
5.3	Kanalunabhängige Kundenkommunikation	10
5.3.1	Kommunikationsabstraktion und Nachrichtenerzeugung	10
5.3.2	E-Mail-Versand und mandantenspezifische SMTP-Konfiguration . .	10
5.3.3	SMS und WhatsApp über Twilio: Technologieauswahl und Integration	10
5.4	Mandantenfähigkeit und Sicherheitskonzept	10
5.4.1	Mandantenkontext und Datenisolation	10
5.4.2	Authentifizierung der DISPO-Schnittstelle	10
5.4.3	Zugriffsschutz des Dashboards	10
5.4.4	Tokenbasierter Kundenzugriff	10
5.4.5	Schutz sensibler Konfigurationsdaten	10

5.5	Abfragemodell für die Disposition	10
5.5.1	Aufbau des leseseitigen Abfragemodells	10
5.5.2	Tourübersicht, dynamische Filterung, Gruppierung und Paginierung	10
5.5.3	Auftragsdetails und Avisierungshistorie	10
5.6	Lieferverfolgung und Geocoding	10
5.6.1	Trackingdaten und Fahrerposition	10
5.6.2	Zielkoordinaten und Geocoding	10
5.6.3	Bereitstellung über die Kundenschnittstelle	10
5.7	Technische Konfiguration und Integrationsumgebung	10
5.7.1	Externe Konfiguration und Laufzeitprofile	10
5.7.2	Lokale Infrastruktur und Simulation externer Systeme	10
6	Frontend – Konzeption und Implementierung	10
6.1	Einordnung des Frontends in das Gesamtsystem	10
6.2	Frontend-Architektur und technische Grundlagen	10
6.2.1	Technologieauswahl	10
6.2.2	Seiten-, Komponenten- und Routingstruktur	10
6.2.3	API-Anbindung und Zustandsverwaltung	10
6.2.4	Responsive Design und wiederverwendbare UI-Komponenten	10
6.3	Avisierungsdashboard	10
6.3.1	Geschützter Dashboard-Zugang	10
6.3.2	Tourübersicht und Statusdarstellung	10
6.3.3	Suche, Filterung und Paginierung	10
6.4	Auftragsdetailansicht und erneute Avisierung	10
6.4.1	Auftragsdaten und Avisierungshistorie	10
6.4.2	Erneute Avisierung, Validierung und Fehlerrückmeldung	10
6.5	Bestätigungsseite für Kunden	10
6.5.1	Tokenbasierter Zugriff und Lieferinformationen	10
6.5.2	Bestätigung, Ablehnung und Kundenkommentar	10
6.5.3	Abgeschlossene, abgelaufene und fehlerhafte Anfragen	10
6.6	Live-Tracking	10
6.6.1	Verfügbarkeit und Datenfluss	10
6.6.2	Karten- und Positionsdarstellung	10
6.6.3	Aktualisierung und Fehlerbehandlung	10
7	Test und Validierung	10
7.1	Teststrategie und Testumgebung	10
7.1.1	Testziele, Testebenen und Auswahl der Prüfschwerpunkte	11
7.1.2	Testwerkzeuge und reproduzierbare Testumgebung	11
7.2	Validierung des Avisierungsservice	11
7.2.1	Domänen- und Anwendungslogik	11
7.2.2	REST-Verträge, Eingabvalidierung und Fehlerantworten	11
7.2.3	Authentifizierung, Sitzungsverwaltung und Mandantenisolation	11
7.2.4	Persistenz- und Abfragemodell	11
7.2.5	Workflow-, Wiederholungs- und Nebenläufigkeitsszenarien	11
7.2.6	Ausgehende Schnittstellen und Rückkanäle	11
7.3	End-to-End-Tests	11
7.3.1	Einordnung der Szenariogruppen	11
7.3.2	Versand und Bereitstellung der Anfrage	12

7.3.3	Kundenzugriff und Anzeige der Lieferdaten	12
7.3.4	Kundenreaktion und Statusänderung	12
7.3.5	Tracking und lieferbezogene Transparenz	13
7.3.6	Problemfälle und Fehlerbehandlung	13
7.3.7	Disposition und erneute Avisierung	13
7.3.8	Bewertung der End-to-End-Testabdeckung	14
8	Projektmanagement und Projektreflexion	14
8.1	Projektmanagement	14
8.1.1	Projektphasen	14
8.1.2	Projektplan	14
8.1.3	Projektaufwand	15
8.2	Projektreflexion	15
8.2.1	Herausforderungen	15
8.2.2	Lessons Learned	16
9	Projektresultate	16
9.1	Gesamtergebnis	16
9.2	Grenzen des Projektergebnisses	16
9.3	Fazit und Ausblick	16
10	Nutzung von Künstlicher Intelligenz	16
11	Verwendete Hilfsmittel	16

1 Einleitung

1.1 Motivation und Problemstellung

Ausgangspunkt des Projekts war die Beobachtung, dass die Rückbestätigung von Lieferterminen in weiten Teilen manuell erfolgte und dadurch mit hohem Abstimmungsaufwand verbunden war. Informationen mussten mehrfach weitergegeben, Rückmeldungen manuell ausgewertet und Statusänderungen separat dokumentiert werden.

Ziel der Projektarbeit war es deshalb, einen strukturierten digitalen Prozess zu entwickeln, der sowohl die fachliche Kommunikation mit dem Kunden als auch die technische Weiterverarbeitung im System unterstützt.

1.2 Zielsetzung

1.3 Methodische Vorgehensweise

2 IST-Analyse

2.1 Bestehender Avisierungsprozess

Im bisherigen Ablauf wurden Liefertermine durch die Disposition vorbereitet und anschließend auf direktem Weg an die Kunden kommuniziert. Rückmeldungen erfolgten je nach Situation per E-Mail, telefonisch oder über informelle Zwischenabsprachen. Die eingehenden Informationen mussten anschließend manuell ausgewertet und an weitere Beteiligte weitergegeben werden.

Eine durchgängige digitale Verarbeitung des Prozesses war dadurch nur eingeschränkt möglich. Insbesondere bei mehreren gleichzeitigen Lieferungen stieg der Koordinationsaufwand deutlich an.

2.2 Bestehende Kommunikationskanäle

2.3 Schwachstellen und Optimierungspotenziale

Die wesentlichen Schwachstellen lagen in der fehlenden Standardisierung der Rückmeldungen und in den Medienbrüchen zwischen Kommunikation, Bearbeitung und Dokumentation. Daraus ergaben sich Verzögerungen, erhöhter Abstimmungsbedarf und ein größeres Risiko für Missverständnisse im weiteren Ablauf.

Zusätzlich fehlte eine einheitliche Statussicht, sodass der aktuelle Bearbeitungsstand eines Liefertermins nicht jederzeit direkt nachvollzogen werden konnte.

2.4 Funktionale Anforderungen

ID	Beschreibung
A1	Das System versendet einen Bestätigungslink über E-Mail, SMS oder WhatsApp.
A2	Der Kunde kann über eine Bestätigungsseite auf den vorgeschlagenen Liefertermin reagieren.
A3	Nach Eingang der Kundenrückmeldung wird der Status automatisch im System aktualisiert.
A4	Bei bestätigten Lieferungen kann am Liefertag eine Fahrerkartenansicht bereitgestellt werden.
A5	Die Disposition erhält über ein Dashboard eine Übersicht aller relevanten Vorgänge und Statuswerte.

2.5 Nichtfunktionale Anforderungen

3 SOLL-Konzept

3.1 Zielbild des Avisierungsprozesses

3.2 SOLL-Prozess

3.2.1 Prozessablauf

Im SOLL-Prozess wird die Kommunikation mit dem Kunden standardisiert ausgelöst und systemseitig begleitet. Nach Eingang eines relevanten Vorgangs wird automatisch eine Benachrichtigung mit einem personalisierten Link versendet. Der Versand kann je nach hinterlegtem Kommunikationsweg über E-Mail, SMS oder WhatsApp erfolgen. Über diesen Link gelangt der Kunde auf eine Bestätigungsseite, auf der er strukturiert auf den vorgeschlagenen Liefertermin reagieren kann.

Diese Rückmeldung wird automatisiert verarbeitet, in einen Status überführt und den beteiligten Stellen ohne manuelle Zwischenschritte bereitgestellt. Dadurch entsteht ein durchgängiger Prozess von der Auslösung bis zur Anzeige des aktuellen Bearbeitungsstands. Erfolgt eine Bestätigung, kann am Tag der Lieferung zusätzlich eine Fahrerkartenansicht bereitgestellt werden, um den Lieferkontext für den Kunden transparent darzustellen. Parallel dazu behält die Disposition über ein Dashboard den Überblick über offene, bestätigte oder problematische Vorgänge.

3.2.2 BPMN-Modell

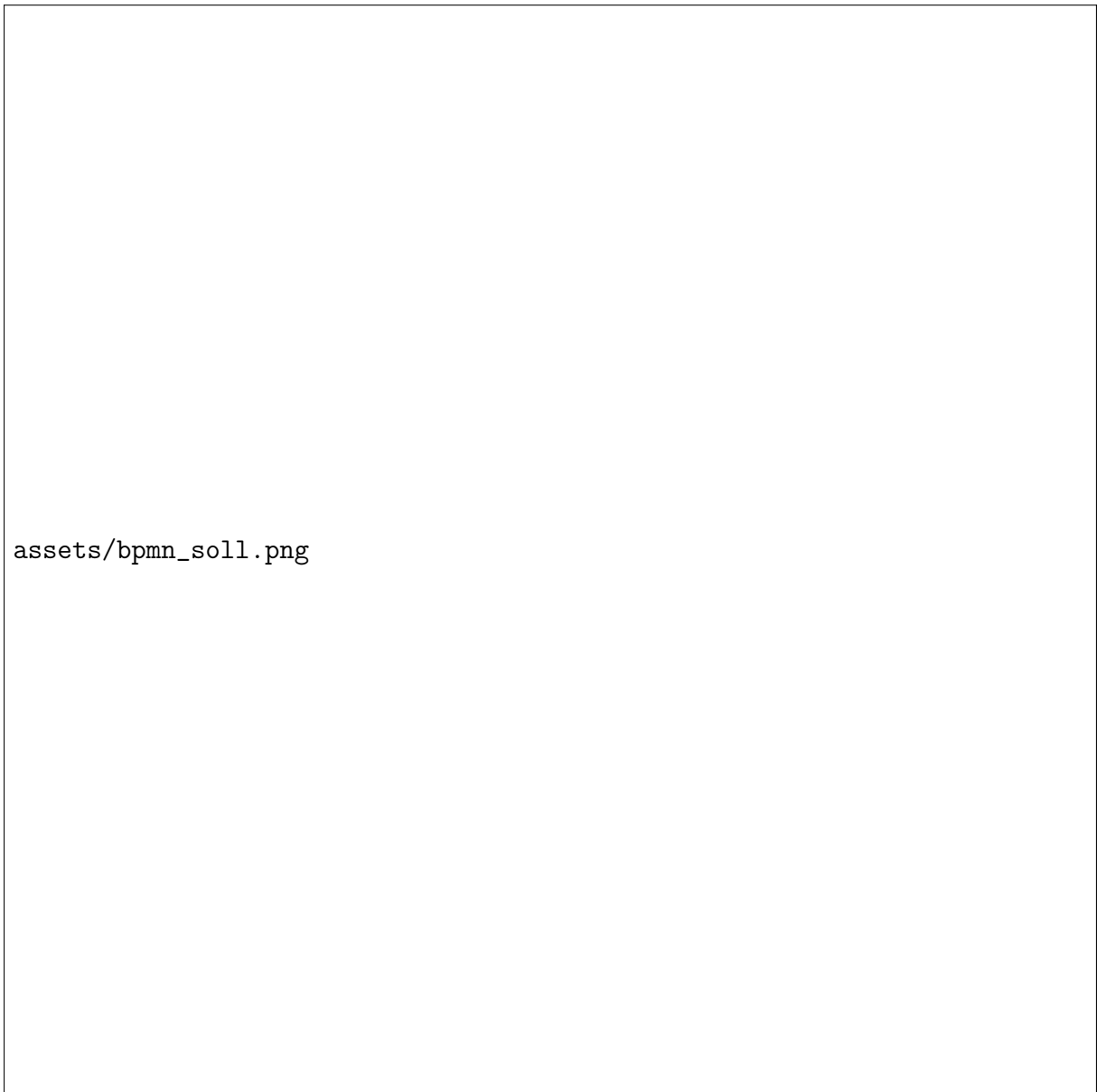


Abbildung 1: BPMN-Modell des SOLL-Prozesses

4 Systemarchitektur

4.1 Gesamtarchitektur



Abbildung 2: Systemarchitektur

4.2 Systemkomponenten

Das Gesamtsystem besteht aus mehreren miteinander kommunizierenden Komponenten. Das Frontend stellt die Benutzeroberflächen für die Disposition und die Kunden bereit. Das Backend übernimmt die fachliche Verarbeitung, stellt Schnittstellen bereit und koordiniert die Kommunikation mit weiteren Systemkomponenten.

Die Workflow Engine unterstützt die Modellierung und Steuerung des Rückbestätigungsprozesses. Persistente Informationen wie Lieferdaten, Rückmeldungen und Statuswerte werden in einer Datenbank gespeichert. Zusätzlich werden externe Dienste für die verschiedenen Kommunikationskanäle angebunden.

4.3 Schnittstellen und Interaktionen

Die Systemkomponenten sind über definierte Schnittstellen miteinander verbunden. Lieferaufträge werden aus dem Dispositionskontext an das Backend übergeben und dort verarbeitet. Das Backend koordiniert die weitere Workflow-Steuerung, persistiert relevante Zustände und stellt die benötigten Daten für die Benutzeroberflächen bereit.

Für die Kommunikation mit dem Kunden werden externe Kommunikationsdienste angebunden. Kundenrückmeldungen werden wiederum vom Backend verarbeitet und stehen anschließend der Disposition über das Dashboard zur Verfügung.

5 Backend des Avisierungsservice – Konzeption und Implementierung

5.1 Architektur- und Domänenentwurf

5.1.1 Architekturprinzipien, Verantwortungsgrenzen und Abhängigkeitsrichtung

5.1.2 Domänen- und Zustandsmodell

5.1.3 Persistenzmodell und Schemaentwicklung

5.2 Technische Umsetzung des Avisierungs- und Rückbestätigungsprozesses

5.2.1 Übernahme von Avisierungsaufträgen, Validierung und Prozessstart

5.2.2 Dauerhafte Workflow-Steuerung mit Camunda

5.2.3 Kundenreaktion und Fristablauf

5.2.4 Erneute Avisierung und Ablösung veralteter Anfragen

5.2.5 Statusrückmeldung an DISPO

5.2.6 Fehlerbehandlung, Wiederholungen, Nebenläufigkeit und Idempotenzgrenzen

5.3 Kanalunabhängige Kundenkommunikation

5.3.1 Kommunikationsabstraktion und Nachrichtenerzeugung

5.3.2 E-Mail-Versand und mandantenspezifische SMTP-Konfiguration

5.3.3 SMS und WhatsApp über Twilio: Technologieauswahl und Integration

5.4 Mandantenfähigkeit und Sicherheitskonzept

5.4.1 Mandantenkontext und Datenisolation

5.4.2 Authentifizierung der DISPO-Schnittstelle

5.4.3 Zugriffsschutz des Dashboards

5.4.4 Tokenbasierter Kundenzugriff

5.4.5 Schutz sensibler Konfigurationsdaten

5.5 Abfragemodell für die Disposition

5.5.1 Aufbau des leseseitigen Abfragemodells

5.5.2 Tourübersicht, dynamische Filterung, Gruppierung und Paginierung

5.5.3 Auftragsdetails und Avisierungshistorie

5.6 Lieferverfolgung und Geocoding

5.6.1 Trackingdaten und Fahrerposition

5.6.2 Zielkoordinaten und Geocoding

5.6.3 Bereitstellung über die Kundenschnittstelle

5.7 Technische Konfiguration und Integrationsumgebung

Frontend, Backend, Workflow Engine und Kommunikationskanälen getestet.

Ein besonderer Fokus lag dabei auf den Übergängen zwischen den Systemkomponenten, da dort fachliche Fehler oder inkonsistente Status besonders wahrscheinlich sind.

7.1.1 Testziele, Testebenen und Auswahl der Prüfschwerpunkte

7.1.2 Testwerkzeuge und reproduzierbare Testumgebung

7.2 Validierung des Avisierungsservice

7.2.1 Domänen- und Anwendungslogik

7.2.2 REST-Verträge, Eingabevalidierung und Fehlerantworten

7.2.3 Authentifizierung, Sitzungsverwaltung und Mandantenisolation

7.2.4 Persistenz- und Abfragemodell

7.2.5 Workflow-, Wiederholungs- und Nebenläufigkeitsszenarien

7.2.6 Ausgehende Schnittstellen und Rückkanäle

7.3 End-to-End-Tests

Die End-to-End-Tests prüfen die für Kunden sichtbaren Kernabläufe über Frontend, Backend, E-Mail-Versand und DISPO- Rückmeldung hinweg. Sie werden mit Playwright im Chromium-Browser ausgeführt und befinden sich im Verzeichnis `frontend/e2e`.

7.3.1 Einordnung der Szenariogruppen

Die Teststruktur orientiert sich an fachlichen Benutzerabläufen und nicht an einzelnen Komponenten. Dadurch bleiben die Tests nah am tatsächlichen Nutzungsszenario und können leichter gewartet werden.

Die Tests zur Kundenrückbestätigung befinden sich im Verzeichnis `frontend/e2e/confirmation`. Dort sind die Szenarien auf zwei Spec-Dateien verteilt:

- `confirmation.spec.ts` enthält die zentralen Kundenabläufe. Dazu gehören die Anzeige der Lieferdaten, die Bestätigung und Ablehnung eines Termins, das erneute Öffnen eines bereits verwendeten Links, ungültige Links, Fehler beim Absenden einer Rückmeldung sowie die Information, dass Tracking vor dem Liefertag noch nicht verfügbar ist.
- `confirmation-email.spec.ts` prüft die Bereitstellung der Bestätigungs-E-Mail. Dabei wird kontrolliert, ob die E-Mail die relevanten Lieferinformationen und den personalisierten Bestätigungslink enthält.

Die Tests zur Lieferverfolgung befinden sich im Verzeichnis `frontend/e2e/tracking`. Die Datei `tracking.spec.ts` deckt die Anzeige der Tracking-Karte, die Darstellung von Fahrzeug und Zieladresse sowie die Aktualisierung der Fahrerposition ab.

Wiederverwendbare technische Abläufe befinden sich im Verzeichnis `frontend/e2e/helpers`. Diese Hilfsfunktionen reduzieren Duplikate, ohne den fachlichen Ablauf in den Spec-Dateien zu verbergen:

- `confirmation-request.ts` erzeugt eindeutige Testaufträge und stellt wiederverwendbare Lieferdaten bereit.
- `confirmation-flow.ts` fasst die wiederkehrenden Schritte zum Erzeugen einer Anfrage, zum Warten auf den Bestätigungslink und zum Öffnen der Kundenseite zusammen.
- `mailpit.ts` wartet auf die für einen Testkunden bestimmte E-Mail und liest bei Bedarf deren Inhalt aus.
- `dispo-mock.ts` prüft, ob die erwartete Statusrückmeldung, beispielsweise `CONFIRMED` oder `REJECTED`, beim DISPO-Mock eingegangen ist.
- `confirmed-tracking-page.ts` bereitet einen bestätigten Auftrag für Tracking-Szenarien vor.
- `services.ts` prüft vor jedem Test die Erreichbarkeit von Backend, Mailpit und DISPO-Mock.

Die Spec-Dateien beschreiben die fachlichen Abläufe aus Sicht der Kunden. Wiederverwendbare Hilfsfunktionen übernehmen technische Schritte, beispielsweise das Anlegen von Testdaten oder das Auslesen einer E-Mail.

7.3.2 Versand und Bereitstellung der Anfrage

Ein zentraler Testfall erzeugt eine Rückbestätigungsanfrage für einen eindeutigen Testauftrag. Anschließend wird geprüft, ob die zugehörige E-Mail in Mailpit eingeht. Der Inhalt der E-Mail muss die wesentlichen Lieferinformationen enthalten: Kundenname, Lieferadresse, Produkt, Menge, Lieferzeitfenster, Preis sowie den personalisierten Bestätigungslink.

Damit wird der Übergang von der Übernahme eines Avisierungsauftrags bis zur für den Kunden erreichbaren Anfrage abgedeckt. Der Test prüft bewusst nicht die interne Generierung des E-Mail-Templates oder die technische SMTP-Konfiguration im Detail, sondern deren sichtbares Ergebnis.

7.3.3 Kundenzugriff und Anzeige der Lieferdaten

Nach dem Öffnen des Links wird geprüft, ob die Bestätigungsseite die für die Entscheidung erforderlichen Informationen anzeigt. Dazu gehören insbesondere Kundenname, Lieferadresse, Produkt, Bestellmenge, Preis sowie Beginn und Ende des Lieferzeitfensters.

Zusätzlich wird ein ungültiger Link getestet. In diesem Fall darf keine Bestätigungsaktion möglich sein; stattdessen muss eine für den Kunden verständliche Fehlermeldung angezeigt werden. Für bereits verwendete Links wird der bereits erreichte Abschlussstatus angezeigt.

7.3.4 Kundenreaktion und Statusänderung

Die beiden fachlich wichtigsten Kundenreaktionen werden als separate E2E-Szenarien geprüft. Der Kunde kann einen Liefertermin bestätigen oder ablehnen. Nach der Aktion muss eine sichtbare Erfolgsmeldung erscheinen.

Beim Bestätigen wird zusätzlich geprüft, ob der Status `CONFIRMED` an den DISPO-Mock übermittelt wird. Beim Ablehnen wird entsprechend die Rückmeldung `REJECTED` erwartet.

Dadurch wird nicht nur die Anzeige im Frontend, sondern auch die relevante externe Systemwirkung der Kundenentscheidung überprüft.

Für einen Fehler beim Absenden der Kundenreaktion existiert ebenfalls ein Testfall. Die Anwendung zeigt dabei eine konkrete Fehlermeldung an und lässt eine erneute Eingabe zu. Damit wird vermieden, dass ein technischer Übertragungsfehler fälschlich als ungültiger Bestätigungslink dargestellt wird.

7.3.5 Tracking und lieferbezogene Transparenz

Für bestätigte Lieferungen am Liefertag wird geprüft, ob die Live-Tracking-Ansicht verfügbar ist. Der Test erwartet eine sichtbare Karte, eine Fahrzeugposition und eine Zielmarkierung an der Lieferadresse. Außerdem wird geprüft, ob die Zieladresse als für den Kunden verständlicher Text angezeigt wird.

Ein weiterer Test prüft die Aktualisierung der Fahrerposition über die Schaltfläche „Aktualisieren“. Nach der Aktualisierung bleiben Karte und Markierungen sichtbar; der angezeigte Lieferstatus wird aktualisiert.

Für bestätigte Lieferungen vor dem Liefertag wird dagegen keine Karte erwartet. Die Anwendung informiert den Kunden stattdessen darüber, dass Tracking-Informationen erst am Liefertag verfügbar sind.

7.3.6 Problemfälle und Fehlerbehandlung

Die E2E-Tests decken derzeit ungültige Links, bereits verwendete Links sowie Fehler beim Absenden einer Kundenreaktion ab. Diese Fälle sind besonders relevant, da sie direkt in der Kundenansicht sichtbar werden und eine klare Benutzerführung erfordern.

Ein eigener Test für tatsächlich abgelaufene Links ist derzeit noch nicht enthalten. Die Anwendung stellt diesen Zustand als abgelaufene Anfrage dar, jedoch fehlt in der Testumgebung eine kontrollierbare Zeitquelle oder ein geeigneter Test-Endpunkt, um einen zuvor gültigen Link reproduzierbar ablaufen zu lassen. Dieser Testfall sollte ergänzt werden, sobald eine solche Testmöglichkeit vorhanden ist.

Auch ein sichtbarer Fehler beim Aktualisieren der Trackingdaten wird derzeit nicht getestet, da die Anwendung diesen Fehler bewusst ohne Benutzerhinweis behandelt und die bestehende Karte beibehält.

7.3.7 Disposition und erneute Avisierung

Für die Disposition ist im Backend ein geschützter Dashboard-Zugang mit Tourübersicht, Auftragsdetails und erneuter Avisierung vorgesehen. Die aktuelle Frontend-Implementierung verwendet jedoch noch nicht die tatsächlich vorhandenen Dashboard-Endpunkte und implementiert den erforderlichen Zugangscode- und Sitzungsaufbau nicht vollständig.

Aus diesem Grund wurden für das Dashboard noch keine belastbaren Playwright-End-to-End-Tests als erfolgreich bewertete Testfälle aufgenommen. Tests würden derzeit lediglich Mock-Verhalten oder einen Ladefehler prüfen, nicht aber einen realen Dispatcher-Ablauf.

Nach Abschluss der Frontend-Backend-Integration sollten mindestens folgende Dispatcher-Szenarien ergänzt werden: Zugang über einen Dashboard-Link, Anzeige von Touren und

Aufträgen, Öffnen eines Auftrags mit Avisierungshistorie sowie erneutes Versenden einer Rückbestätigungsanfrage. Als besonders wichtiger systemübergreifender Test sollte anschließend geprüft werden, dass die erneute Avisierung einen neuen nutzbaren Kundenlink erzeugt.

7.3.8 Bewertung der End-to-End-Testabdeckung

Die vorhandenen End-to-End-Tests decken die wesentlichen kundenseitigen Kernabläufe ab: E-Mail-Bereitstellung, Anzeige der Lieferdaten, Bestätigung und Ablehnung, Statusrückmeldung an DISPO sowie Live-Tracking am Liefertag.

Nicht Bestandteil der Playwright-Tests sind bewusst Implementierungsdetails des Backends. Dazu zählen beispielsweise DTO-Validierungen, Datenbankabfragen, Token-Erzeugung, Mandantenisolation, Workflow-Retries und die Berechnung von Tracking-Distanzen. Diese Aspekte werden auf Unit- und Integrationstestebene geprüft.

Die Kundenansicht ist damit für die zentralen sichtbaren Geschäftsabläufe sinnvoll abgedeckt. Eine vollständige End-to-End-Abdeckung des Gesamtsystems setzt zusätzlich die Fertigstellung der Dashboard-Integration voraus.

8 Projektmanagement und Projektreflexion

8.1 Projektmanagement

8.1.1 Projektphasen

Das Projekt wurde in klar abgegrenzte Phasen unterteilt, um Analyse, Konzeption, Umsetzung und Validierung strukturiert durchführen zu können. Die Übergänge zwischen den Phasen waren teilweise iterativ, da Erkenntnisse aus der Implementierung wieder in die Ausgestaltung des SOLL-Prozesses eingeflossen sind.

Phase	Zeitraum	Status
Analyse und Anforderungsabstimmung	xx.xx - xx.xx	abgeschlossen
Architektur und Prozessmodellierung	xx.xx - xx.xx	abgeschlossen
Implementierung	xx.xx - xx.xx	abgeschlossen
Integration und Test	xx.xx - xx.xx	abgeschlossen
Abschluss und Ergebnisaufbereitung	xx.xx - xx.xx	abgeschlossen

8.1.2 Projektplan

Der Projektplan diene als gemeinsame Orientierung für inhaltliche Meilensteine, Abstimmungstermine und die Reihenfolge der Umsetzung. Besonders hilfreich war die frühe Sichtbarkeit von Abhängigkeiten zwischen Prozessmodell, Schnittstellen und Benutzeroberfläche.

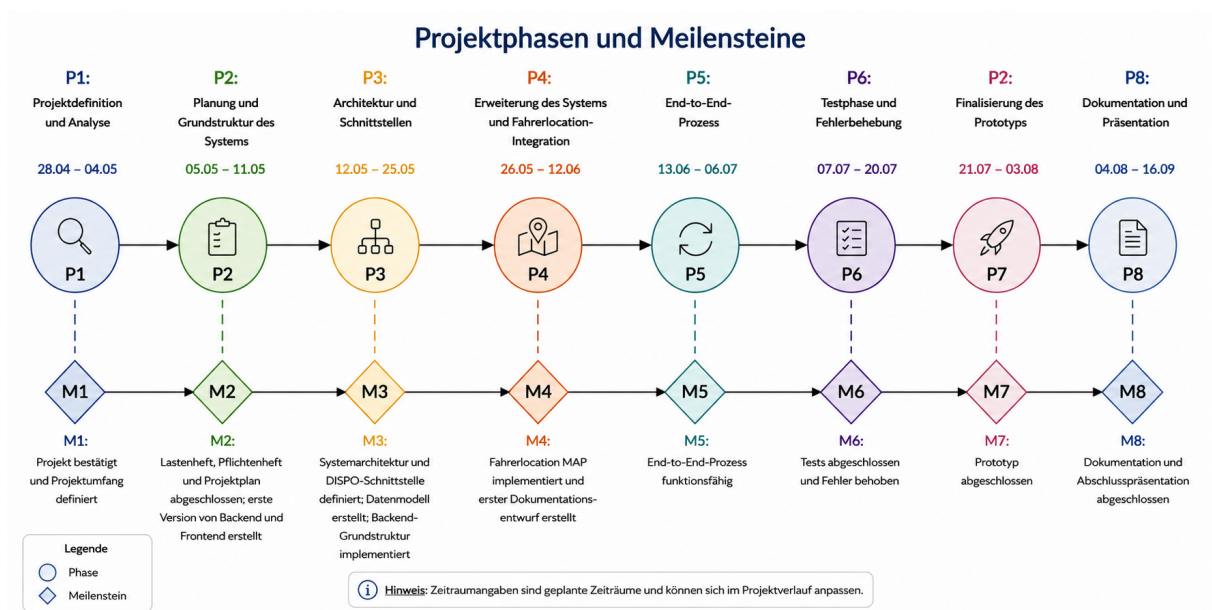


Abbildung 3: Projektplan

8.1.3 Projektaufwand

Der dokumentierte Aufwand verteilt sich auf Analyse, Konzeption, Implementierung, Test und Abstimmung mit dem Auftraggeber. Die Tabelle dient als Beispielstruktur zur Erfassung der geleisteten Stunden und kann mit den tatsächlichen Projektdaten vervollständigt werden.

Datum	Person	Tätigkeit	Stunden
xx.xx.2026	Name 1	Anforderungsanalyse und Recherche	4
xx.xx.2026	Name 2	Backend-Implementierung	6
xx.xx.2026	Name 3	BPMN-Modellierung und Camunda-Integration	5
xx.xx.2026	Name 1	Frontend-Umsetzung und UI-Anpassungen	5
xx.xx.2026	Team	Integrationstest und Ergebnisabstimmung	4

8.2 Projektreflexion

8.2.1 Herausforderungen

Herausfordernd war insbesondere die Abstimmung an den Schnittstellen zwischen fachlicher Prozessbeschreibung und technischer Umsetzung. Bereits kleine Unklarheiten bei Statusübergängen oder Rückmeldewegen hatten direkte Auswirkungen auf mehrere Systemkomponenten.

Auch die Konsistenz zwischen Benutzerführung, Datenmodell und Workflowlogik erforderte wiederholte Abstimmungen.

8.2.2 Lessons Learned

Für vergleichbare Projekte empfiehlt es sich, Statusmodelle, Ereignisse und Verantwortlichkeiten frühzeitig gemeinsam zu definieren und schriftlich festzuhalten. Darüber hinaus hat sich gezeigt, dass ein visuelles Prozessmodell in Kombination mit einer frühen technischen Teilimplementierung sehr hilfreich ist, um Missverständnisse schnell sichtbar zu machen.

9 Projektrgebnisse

9.1 Gesamtergebnis

Die Projektarbeit führte zu einem konsistenten Prototypen, der die wesentlichen Anforderungen des definierten SOLL-Prozesses abbildet.

- Funktionsfähiger Prototyp
- Frontend-Backend-Kommunikation
- Automatisierte Kommunikation über verschiedene Kanäle
- Digitale Rückbestätigung von Lieferterminen
- Statusverwaltung
- Workflow-Steuerung
- Avisierungsdashboard für die Disposition
- Bestätigungsseite für Kunden
- Lieferverfolgung bzw. Fahrerkartenansicht

9.2 Grenzen des Projektergebnisses

9.3 Fazit und Ausblick

Das Projekt konnte sowohl aus technischer als auch aus organisatorischer Sicht erfolgreich abgeschlossen werden. Neben dem entwickelten Prototypen entstand eine belastbare Grundlage für die weitere fachliche und technische Ausarbeitung des Systems.

Die Kombination aus Prozessmodellierung, Webanwendung und automatisierter Kommunikation erwies sich als geeigneter Ansatz für die Digitalisierung des betrachteten Anwendungsfalls.

10 Nutzung von Künstlicher Intelligenz

11 Verwendete Hilfsmittel